Imitation Learning with Latent Market Parameters in Continuous-Time Portfolio Control

Mark Frankli · Wilfrid Laurier University, Department of Mathematics

SSC Student Conference 2026 · May 2026

Motivation & Problem Setting

Portfolio optimization is a fundamental task in quantitative finance. Traditional methods rely on **strong assumptions** and offer limited flexibility, while Machine Learning, and Reinforcement Learning (RL) in particular, have demonstrated great success in financial applications.

Goal: Distribute (and periodically reallocate) wealth across n assets to maximize a utility function.

We study the $n = 2$ case:

- **Risky asset:** mean return μ, volatility σ
- **Risk-free asset:** constant return r

Research Questions

1. Can RL-based algorithms recover the expert policy?
2. Under what circumstances is imitation learning (IL) a *better* choice than RL?
3. How can IL enhance well-established RL algorithms such as PPO?

Our Contributions

- **Contribution 1.** Rigorous comparison of IL algorithms (BC, DAgger) to traditional optimal control solutions.
- **Contribution 2.** An RNN-based IL algorithm that **outperforms the Merton baseline** by implicitly inferring latent market parameters from observed trajectories.
- **Contribution 3.** Hot-starting PPO with the IL-learned policy **stabilizes training** and improves performance (transfer learning).

Imitation Learning Algorithms

Behavior Cloning (BC):

1. Initialize $\mathcal{D}$ with expert trajectories
2. $\max_{\pi_\theta} \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log \pi_\theta(a \mid s)]$ via gradient descent

DAgger:

1. Initialize $\mathcal{D} \leftarrow \emptyset$, $\hat{\pi}_1 \leftarrow$ random policy
2. For $i = 1, \ldots, N$:
   - 2.1 Mixture policy: $\pi_i = \beta_i \pi^* + (1 - \beta_i)\hat{\pi}_i$
   - 2.2 Roll out π_i; label visited states with expert actions $\pi^*(s)$; add to $\mathcal{D}$
   - 2.3 Train $\hat{\pi}_{i+1}$ on $\mathcal{D}$

π_t: fraction of wealth in risky asset at time t.

Jump-Diffusion Model

The second expert model allows occasional price shocks:

$$\frac{dS_t}{S_t} = \mu_{\text{eff}}\, dt + \sigma\, dW_t + (J - 1)\, dN_t$$

No closed-form solution exists; we use the second-order approximation:

$$\pi^* \approx \frac{\mu - r}{\gamma(\sigma^2 + \lambda\, \mathbb{E}[(J-1)^2])}$$

Recovering even a constant policy is non-trivial, making DAgger essential in this setting.

Merton Model (Expert Policy 1)

Merton's stochastic control model is the first expert policy.

Wealth dynamics:

$$\frac{dX_t}{X_t} = [(\mu - r)\pi_t + r]\, dt + \pi_t \sigma dW_t$$

Objective:

$$\max\ \mathbb{E}\left[\int_0^T U(C(t))\, dt + B(X_T)\right]$$

Under **CRRA utility** $U(x) = x^{1-\gamma}/(1-\gamma)$, the optimal allocation has a **closed form**:

$$\boxed{\pi_t^* = \frac{\mu - r}{\gamma\sigma^2}}$$

Returns are approximated as $R_t \approx \mu\Delta t + \sigma\sqrt{\Delta t} Z_t$, $Z_t \sim \mathcal{N}(0,1)$, and wealth evolves as $X_{t+1} \approx X_t\,(1 + r\Delta t + \pi_t(R_t - r\Delta t))$.

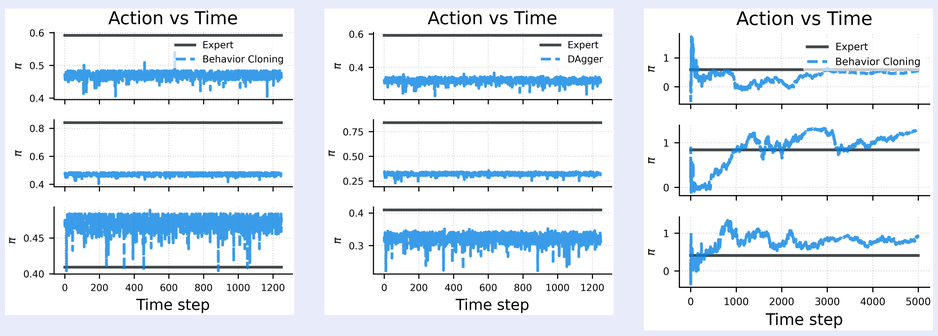
Trajectory-Dependent Setting

When μ and σ are *fixed*, recovering the optimal policy is straightforward. We extend to a harder, more realistic setting:

$$\mu_k \sim \mathcal{N}(\mu,\, \sigma_d^2), \qquad \sigma_k \sim \text{LogNormal}(\log\sigma,\, \sigma_d^2)$$

drawn once per trajectory k and *unobserved* by the agent. States that only contain R_t or X_t are insufficient for feed-forward networks in this partial-observability setting, so we first augment the state with running statistics $\hat{\mu}_t, \hat{\sigma}_t$, but predictions remain too volatile, motivating our RNN approach.

Feed-Forward NN Approximations



Left: BC on 5-year trajectories (POMDP). **Centre:** DAgger on 5-year trajectories (POMDP). **Right:** BC on 20-year trajectories with sufficient statistics $\hat{\mu}_t, \hat{\sigma}_t$. Including sample statistics improves tracking but policies remain volatile.

Proposed RNN Policy

The RNN reads the *full trajectory history* to implicitly infer latent parameters (μ_k, σ_k), then outputs a normal **distribution** over allocations:

- Outputs mean and variance *over actions*
- Loss function: Gaussian negative log-likelihood
- Each trajectory is compressed to one state–action pair

RNN Performance – Merton Model

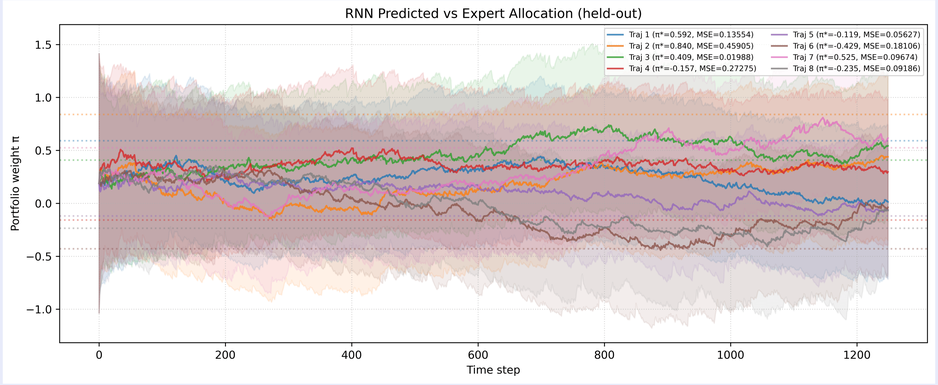
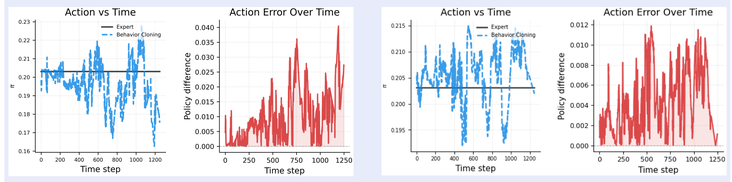


Figure: RNN predicted vs. expert allocation on held-out trajectories. Each line is one trajectory; shaded bands show $\pm 1\sigma$ confidence. Dotted horizontal lines mark the per-trajectory π^*. The RNN policy is better than the feed-forward policy, but the approximations can still be imprecise.

BC vs DAgger - Jump-Diffusion



Left pair: BC action and error. **Right pair:** DAgger action and error. DAgger substantially reduces both absolute policy error and error accumulation over time.

RNN vs. Naive Plug-in Policy

When (μ_k, σ_k) are unobserved, the **naive policy** plugs sample mean and variance into the Merton formula. The **RNN policy** implicitly infers them from the trajectory.

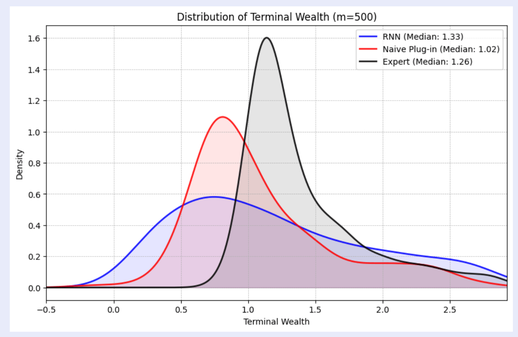


Figure: Terminal wealth distributions ($m = 500$). RNN (blue) has a tighter, higher-median distribution than the naive plug-in (red).

Policy	Mean	Median	Std	CRRA utility
RNN	2.09	1.33	2.28	-36.5
Naive	2.88	1.02	15.43	-2.5×10^{45}
Expert	2.00	1.26	3.63	-0.10

The RNN achieves dramatically higher CRRA utility than the naive policy, at the cost of a slightly heavier left tail.

Pre-training PPO with IL

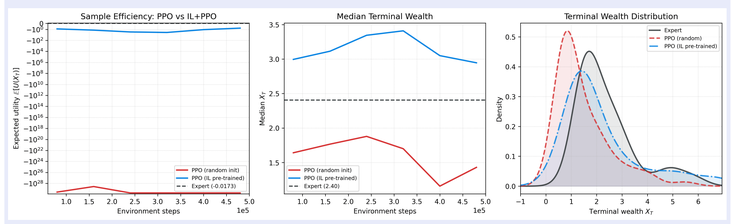


Figure: **Left:** Sample efficiency (log-scale). **Centre:** Median terminal wealth. **Right:** Terminal wealth distribution. IL pre-trained PPO (blue) converges faster and achieves higher median wealth; randomly initialized PPO (red) has near-infinite negative CRRA utility.

Conclusions

1. **Can RL recover the expert?** Yes, IL suffices for the *static* Merton and jump diffusion setting. In the trajectory-based setting, RNNs are required and the approximations can still be imprecise.
2. **When is IL better?** IL is a better choice when optimal control requires parameters that are unobservable. Under *distribution shift* (e.g. volatility regime change), the analytical solution is more robust if true parameters are known.
3. **IL + PPO:** IL pre-training **stabilizes PPO training** and yields higher median terminal wealth, confirming the value of transfer learning in RL-based portfolio optimization.